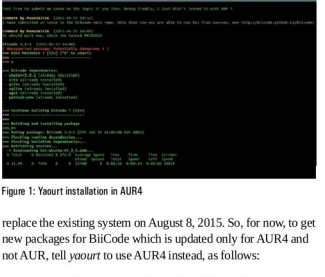

Figure 1: Yaourt installation in AUR4

replace the existing system on August 8, 2015. So, for now, to get new packages for BiiCode which is updated only for AUR4 and not AUR, tell *yaourt* to use AUR4 instead, as follows:

```
yaourt --aur-url https://aur4.archlinux.org biicode
```

After the transition is complete, there is no need to supply the *—aur-url* parameter. Now that I have the latest BiiCode version, we can proceed further. BiiCode will install the necessary tools for development, but as a developer, it is considered good practice to have all the necessary tools pre-installed and configured.

Next, check if the necessary building tools like the compiler (GCC) and Cmake are installed. If not, you can either install them manually or BiiCode can install them automatically for you.

```
bii setup:cpp
```

Note: Do manual installation only when you want to do so, or if the automatic set-up fails.

First, install the necessary tools for basic building. In Arch Linux, this is called *base-devel*. Ubuntu has build-essentials as an equivalent, followed by Cmake.

```
sudo pacman -S base-devel cmake
```

We now have all the necessary tools.

Usage

Let's try BiiCode by using the same example that is written in BiiCode docs. Use Google's GTest library without installing it, as follows:

```
bii init unit_test -L #Creates an empty  project inside a
folder named unit_test
cd unit_test # cd into the testing project folder
vim main.cpp #Create and edit the main.cpp , use any editor
you love
```

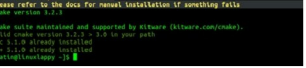

Figure 2: Biicode CPP check

Figure 3: BiiCode build output

Now, create a file named *main.cpp* using your favourite text editor and place the following code inside it— the same example can be found at *http://docs.biicode.com/c++/gettingstarted.html#create-your-first-project* for easy pasting.

```
# include "google/gtest/include/gtest/gtest.h"
int sum(int a,int b) { return a+b;}
TEST(Sum,Normal)
{
EXPECT_EQ(5,sum(2,3));
}
int main(int argc, char **argv)
{
testing::InitGoogleTest(&argc,argv);
return RUN_ALL_TESTS();
}
```

The BiiCode method of resolution is quite intuitive. Like in a normal C++ project, we include a library by its name, followed by the sub-module.

```
   // CPP way
# include <Library_Name/Submodule.h/hpp>
 //Example
# include <opencv2/highgui.hpp>
```

BiiCode header files give you a hint of the project's location and are an easy way for Biicode to resolve project dependencies.

According to BiiCode docs, "The *#include* is composed as *#include <biicode_user>/<block_name>/path/to/file*. In this case, it #includes the *include/gtest/gtest.h* header from the GTest block and user google."

So if I publish a project under the block named 'example', with many files, it will be available as:

```
# include "jatindhankhar/example/filename_with_path"
```